

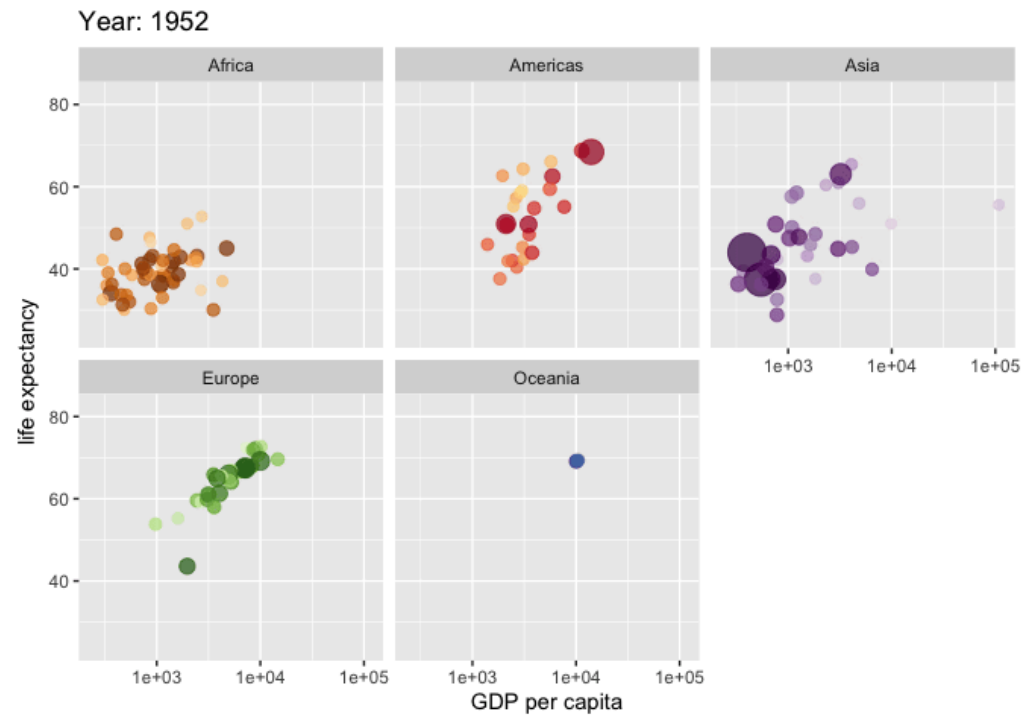
Introduction to data visualization with `ggplot2()`

Pablo E. Gutiérrez-Fonseca, PhD

BIOL-4994 & BIOL-4991 | BIOL-6994 & BIOL-6997
Fall 2025

2025-Aug-01 (updated: 2025-Oct-27)





What is `ggplot2`?

- An implementation of the [Grammar of Graphics](#) proposed by **Leland Wilkinson (2005)**.
- Developed by [Hadley Wickham](#) while a graduate student at Iowa State University. (*He still maintains it!*)
- Represents a **third graphics system** for R — alongside **base** and **lattice** graphics.

What is Grammar of Graphics?

- Created by **Leland Wilkinson (2005)** to define the **fundamental building blocks** of statistical graphics.
- The grammar provides a **structured way** to describe how plots are built.
- Every graphic is made by combining independent components:
 - **Data** → what you want to visualize
 - **Aesthetics (aes)** → mapping of variables to color, shape, size - **Geoms** → visual marks (points, lines, bars)
 - **Stats** → statistical summaries or transformations
 - **Scales, Coords, Facets, Theme** → control layout and appearance

How `ggplot2` builds on the Grammar?

- Built on the [Grammar of Graphics](#) (the "gg" in `ggplot2`) it provides a **consistent system** for creating and customizing plots.
- 💡 The **Grammar of Graphics** is like **language grammar**: just as learning a few verbs, nouns, and adjectives lets you form countless sentences, learning the basic `ggplot2` **building blocks** lets you create hundreds of different plots without memorizing each one.
- The key idea:
 - separate **what** is graphed (the data)
 - from **how** it is graphed (the visualization).

Why `ggplot2`?

- `ggplot2` is one of the most widely used packages for data visualization in R.
- Its **thoughtful defaults** make it easy for beginners to produce clear, elegant, and informative graphs with **minimal, readable code**.

Why `ggplot2`?

- Key strengths:
- 💪 **Flexible and powerful**
 - Build virtually any type of visualization by layering components.
- 🖼️ **Publication-quality graphics**
 - Create clean, elegant, and customizable figures with minimal effort.
- 🔗 **Strong integration with the tidyverse**
 - Works naturally with `dplyr`, `tidyr`, and other tidyverse tools for a smooth data-to-plot workflow.

Why `ggplot2`?

- Together with other [extensions](#), it provides even more powerful tools for advanced visualization.
- Be aware there are other visualization tools as well, such as:
 - [tidyplots](#): useful for simplified and structured visualization workflows
 - [plotly](#): for interactive plots
 - [r-charts](#): for exploring diverse chart types with reproducible ggplot2 code

What makes `ggplot2` special?

- Layered approach to plotting
- You build plots step-by-step:
 - `data` → `aesthetics` → `geoms` → `scales/themes` (optional)
- Each component is added as a separate layer using the `+` operator

Why it matters:

1. Few code changes = very different plots
 2. Easy to reuse and adapt code
 3. Extensions unlock even more plot types
- 💡 Works best with tidy data (rows = observations, columns = variables)

Basic Structure of a `ggplot2` plot

To build a plot in **ggplot2**, we start with the data and then *add layers*.
Each line below represents a component that you can include to customize your plot.

```
ggplot(data = <DATA>) +           # 1 Specify the dataset to be plotted
  <GEOM_FUNCTION> (               # 2 Choose a geometry (type of plot: points, bars, lines...)
    mapping = aes(<MAPPINGS>), # 3 Map variables to aesthetics (x, y, color, size...)
    stat = <STAT>,             # 4 (Optional) Statistical transformation (e.g., "identity", '
    position = <POSITION>      # 5 (Optional) Adjust position (e.g., dodge, stack, jitter)
  ) +
  <COORDINATE_FUNCTION> +        # 6 (Optional) Change coordinate system (e.g., flip, polar)
  <FACET_FUNCTION> +             # 7 (Optional) Split plot into subpanels by a grouping variable
  <SCALE_FUNCTION> +             # 8 (Optional) Adjust scales (colors, axes, legends)
  <THEME_FUNCTION>               # 9 (Optional) Customize visual style/theme of the plot
```

Minimum requirements in every `ggplot` plot

- To successfully build a plot, three components are required:
 1. Data
 2. Aesthetic mappings (`aes()`)
 3. A Geom
- Everything else is optional — if not specified, ggplot supplies sensible defaults for stat, position, coordinates, facets, scales, and theme.
- Example starting point:

```
ggplot(data = data, aes(x = <var>, y = <var>))
```

Minimum requirements in every `ggplot` plot

```
ggplot(data = <DATA>, mapping = aes(<MAPPINGS>)) + <GEOM_FUNCTION>()
```

1. The **data argument** is used to bind the plot to a specific data frame in `ggplot2`.
2. The **mapping argument** defines the variables mapped to various aesthetics of the plot, e.g. the x and y axis.
 - `aes(x = <var>, y = <var>)`
3. The **geom_function argument** defines the type of plot, e.g. barplot, scatter plot, boxplot.
 - Geometry function names follow the pattern: `geom_X` where X is the name of the geometry. Some examples include `geom_point()`, `geom_bar()`, and `geom_histogram()`.

For example

R Code

Plot

```
ggplot(data = mpg, aes(x = displ, y = hwy)) +  
  geom_point()
```

Before starting



ggplot2 **versus** ggplot

- People are sometimes confused between `ggplot2` and `ggplot`. The former refers to the **name of the package**, while the latter refers to the **function that you run**.
- When you run the `ggplot()` function, it plots directly to the Plots tab in the Navigation Pane (lower right). Alternatively, you can assign a plot to an R object, then call `print()` to view the plot in the Navigation Pane.

Global versus local aesthetic mappings

- Aesthetic mappings can be specified globally in the `ggplot()` function or locally within individual geoms.
- **Global mappings** apply to all geoms in the plot.
 - Example: `ggplot(data = mpg, aes(x = displ, y = hwy)) + geom_point() + geom_smooth()`
- **Local mappings** apply only to the specific geom they are defined in.
 - Example: `ggplot(data = mpg) + geom_point(aes(x = displ, y = hwy)) + geom_smooth(aes(x = displ, y = hwy))`

Arguments for aes() function

- The `aes()` function takes many potential arguments each of which specifies the aesthetic we are mapping onto a variable:
 - x: What is shown on the x-axis.
 - y: What is shown on the y-axis.
 - label: What is shown as text in plot (when using `geom_text()`)

`ggplot2`: common functions, aesthetics, and geoms



Color related aesthetics:

- `color` =: What is shown by color, where color refers to the color of a point or a line (e.g. lines around a histogram).
- `fill` =: What is shown by fill, where fill refers to the color of an area (e.g. area inside a histogram).
- `alpha` =: What is shown by the transparency of a line or point or area.

Differentiation related aesthetics: linestyle, size, shape

- `shape` =: What is shown by the shape of our points.
- `linestyle` =: What is shown by the linestyle (e.g. dashed, dotted, etc...).
- `size` =: What is shown by the size of a line or point.

Commonly used geoms

- `geom_histogram()`: Makes a histogram.
- `geom_density()`: Makes a density plot.
- `geom_point()`: Makes points - ideal for a scatterplot.
- `geom_jitter()`: Makes jittered points - ideal for showing data when x is categorical or discrete.
- `geom_col()`: or `geom_bar()`: Makes a barplot from count data `geom_col()`, or from all observations `geom_bar()`.
- `geom_line()`: Connect observations with a line.

Faceting

- Faceting allows us to use the concept of small multiples ([Tufte 1983](#)) to highlight patterns.
- For one faceted variable: `facet_wrap(~ <var>, ncol = )`
- For two faceted variable: `facet_grid(<var1>~ <var2>)`, where one is shown by rows, and is shown by columns.

